📚 ספר המערכת - לומדת ענף 71

מדריך המשתמש והמפתח המלא

📖 תוכן עניינים

1. מבוא
2. התחלה מהירה
3. ארכיטקטורה וטכנולוגיות
4. מדריך התלמיד
5. מדריך המנהל
6. עורך טקסט עשיר
7. ניהול קבצים
8. מדריך מפתח
9. שאלות נפוצות

מבוא

מה זו לומדת ענף 71?

לומדת ענף 71 היא מערכת למידה דיגיטלית מודרנית ומלאה לניהול קורסים, פרקים, וחומרי לימוד.

עבור תלמידים:

- 👩‍🎓 צפייה בקורסים מסודרים
- 📖 קריאת פרקים עם תוכן עשיר (טקסט, תמונות, קבצים)
- 📊 מעקב התקדמות בקורסים
- ✅ פתרון בחינות וקבלת ציונים
- 📥 הורדת חומרי לימוד (PDF, Word, Excel וכו')

עבור מנהלים:

- 📚 יצירה וניהול קורסים מלא
- 📝 כתיבת תוכן עם עורך טקסט WYSIWYG
- 🖼️ העלאת תמונות וקבצים
- ❓ יצירה וניהול של בחינות ושאלות
- 👥 ניהול משתמשים וגישות

תכונות כלליות:

- ✨ עיצוב Udemy-style מודרני
- 🌙 מצב יום/לילה (Light/Dark)
- 🇮🇱 תמיכה מלאה בעברית (RTL)
- 📱 Responsive design - עובד על כל המכשירים
- 🔒 אבטחה - הצפנת סיסמאות, access control

התחלה מהירה

שלב 1: דרישות מקדימות

Hardware:

- 💻 מחשב עם Windows/Mac/Linux
- 💾 4GB RAM מינימום
- 🔗 חיבור אינטרנט (להורדת dependencies)

Software:

- 🟢 Node.js 16+ (מ-https://nodejs.org/)
- 🐘 PostgreSQL (מ-https://www.postgresql.org/)
- 🌐 דפדפן עדכני (Chrome, Firefox, Edge)

שלב 2: הפעלת המערכת

1. **פתח את תיקיית הפרויקט:**

```
C:\Users\yairk\OneDrive\שולחן העבודה\rf-learning-system
```

2. **לחץ פעמיים על קובץ** `START.bat`
   - שני חלונות CMD יופעלו (Backend ו-Frontend)
   - הדפדפן יפתח אוטומטית ל-http://localhost:5173
3. **המתן 15 שניות** לשרתים להתחיל

שלב 3: התחברות

בעמוד ההתחברות, בחר את התפקיד שלך:

אם אתה תלמיד:

```
123456789    :ת"ז
password123  :סיסמה
```

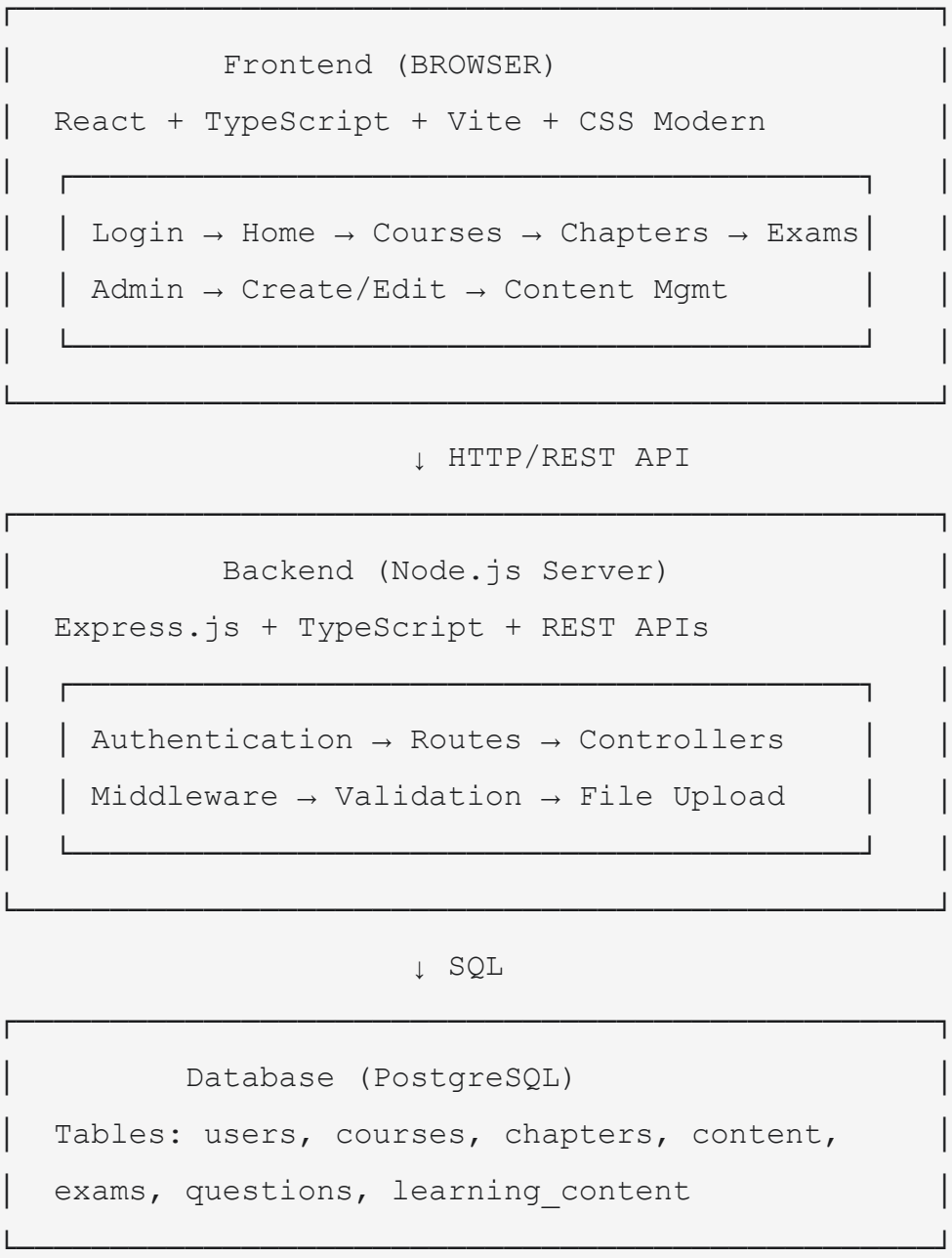
אם אתה מנהל:

```
999999999    :ת"ז
postgres123  :סיסמה
```

ארכיטקטורה וטכנולוגיות

📐 סקירת המערכת

לומדת ענף 71 בנויה בארכיטקטורת **Tier-3 (שלוש שכבות)**:



🛠️ Stack טכנולוגי

Frontend (React + TypeScript)

```
// מנת מהטכנולוגיות:
- React 18 - ספריית UI
- TypeScript - Type safety
- Vite - Build tool מהיר
- React Router - Navigation
- CSS Modern - Styling (Flexbox, Grid)
- Heebo Font - עברית יפה
```

מיקום קבצים:

```
frontend/
├── src/
│   ├── components/           # React components
│   │   ├── Navbar.tsx       # Navigation bar
│   │   ├── RichTextEditor.tsx # WYSIWYG editor
│   │   ├── ContentViewer.tsx # Content display
│   │   └── ...
│   ├── pages/               # Page components
│   │   ├── LoginPage.tsx
│   │   ├── HomePage.tsx
│   │   ├── CoursePage.tsx
│   │   ├── ChapterPage.tsx
│   │   ├── AdminPage.tsx
│   │   └── ...
│   ├── context/             # React Context
│   │   ├── AuthContext.tsx  # Login state
│   │   └── ThemeContext.tsx # Dark/Light mode
│   ├── services/            # API calls
│   │   └── api.ts           # Fetch wrappers
│   ├── App.tsx              # Root component
│   └── App.css              # Global styles
└── package.json             # Dependencies
```

שפות וספריות עיקריות:

- **React** - UI components

- **TypeScript** - Type safety
- **Vite** - Fast build
- **CSS3** - Styling with variables
- **Fetch API** - HTTP requests

Backend (Node.js + Express)

```
// מנת מהטכנולוגיות:
- Node.js - JavaScript runtime
- Express - Web framework
- TypeScript - Type safety
- PostgreSQL - Database
- ts-node - TypeScript execution
- Multer - File uploads
```

מיקום קבצים:

```
backend/
├── src/
│   ├── controllers/          # Business logic
│   │   ├── userController.ts
│   │   ├── courseController.ts
│   │   ├── contentController.ts
│   │   └── ...
│   ├── routes/               # API routes
│   │   ├── users.ts
│   │   ├── courses.ts
│   │   ├── content.ts
│   │   └── ...
│   ├── middleware/           # Authentication, validation
│   │   ├── auth.ts
│   │   └── errorHandler.ts
│   ├── config/               # Configuration
│   │   ├── database.ts       # DB connection
│   │   ├── upload.ts         # File upload settings
│   │   └── ...
│   ├── models/               # Data models
│   ├── server.ts             # Entry point
│   └── db.sql                # Database schema
└── package.json              # Dependencies
```

שפות וספריות עיקריות:

- **Node.js** - Runtime environment
- **Express** - Routing & middleware
- **TypeScript** - Type safety
- **PostgreSQL** - Database
- **Multer** - File handling
- **bcrypt** - Password hashing

Database (PostgreSQL)

Schema - טבלות ויחסים:

```
-- Users table
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name VARCHAR(100),
  id_number VARCHAR(20) UNIQUE,
  password_hash VARCHAR(255),
  role VARCHAR(20), -- 'admin' or 'student'
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Courses table
CREATE TABLE courses (
  id SERIAL PRIMARY KEY,
  title VARCHAR(200),
  description TEXT,
  created_by INT REFERENCES users(id),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Chapters table
CREATE TABLE chapters (
  id SERIAL PRIMARY KEY,
  course_id INT REFERENCES courses(id),
  title VARCHAR(200),
  description TEXT,
  sort_order INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Learning Content table (טקסט, תמונות, קבצים)
CREATE TABLE learning_content (
  id SERIAL PRIMARY KEY,
  chapter_id INT REFERENCES chapters(id),
  type VARCHAR(50), -- 'TEXT', 'RICH', 'IMAGE', 'FILE'
  content TEXT,
```

```
  file_path VARCHAR(500),
  sort_order INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Exams table
CREATE TABLE exams (
  id SERIAL PRIMARY KEY,
  course_id INT REFERENCES courses(id),
  title VARCHAR(200),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Questions table
CREATE TABLE questions (
  id SERIAL PRIMARY KEY,
  exam_id INT REFERENCES exams(id),
  text VARCHAR(500),
  type VARCHAR(50), -- 'SINGLE', 'MULTIPLE', 'FREE_TEXT'
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

:ER Diagram

```
users
  ├── 1 : N ──→ courses (created_by)
  ├── 1 : N ──→ exams (creator)
  └── 1 : N ──→ exam_results (user_id)

courses
  ├── 1 : N ──→ chapters
  └── 1 : N ──→ exams

chapters
  └── 1 : N ──→ learning_content

exams
  └── 1 : N ──→ questions

exam_results
  └── N : M ──→ questions (answers)
```

API Endpoints 🌐

Authentication

```
POST   /api/auth/login          - התחברות
POST   /api/auth/logout         - התנתקות
GET    /api/auth/me             - פרטים על משתמש נוכחי
```

Courses

```
GET    /api/courses             - קבלת כל הקורסים
GET    /api/courses/:id         - קורס ספציפי
POST   /api/courses             - יצירת קורס (admin)
PUT    /api/courses/:id         - עריכת קורס (admin)
DELETE /api/courses/:id         - מחיקת קורס (admin)
```

Chapters

```
GET    /api/courses/:courseId/chapters      - פרקים של קורס
GET    /api/chapters/:id                    - פרק ספציפי
POST   /api/chapters                        - יצירת פרק (admin)
PUT    /api/chapters/:id                    - עריכת פרק (admin)
DELETE /api/chapters/:id                    - מחיקת פרק (admin)
```

Content

```
GET    /api/chapters/:chapterId/content     - תוכן של פרק
POST   /api/content                         - הוספת תוכן (admin)
PUT    /api/content/:id                     - עריכת תוכן (admin)
DELETE /api/content/:id                     - מחיקת תוכן (admin)
```

File Upload

```
POST   /api/upload              - העלאת קובץ
GET    /api/uploads/:filename   - הורדת קובץ
```

Exams & Questions

```
GET    /api/exams/:id           - בחינה ושאלותיה
POST   /api/exams               - יצירת בחינה (admin)
POST   /api/questions            - הוספת שאלה (admin)
POST   /api/exam-results         - שמירת תשובות
GET    /api/exam-results/:id     - התוצאות
```

מדריך התלמיד

👨‍🎓 כמה תלמיד משתמש בלומדה

צעד 1: התחברות

1. הזן ת"ז וסיסמה
2. לחץ "התחברות"
3. תופיע דף הבית עם רשימת קורסים

צעד 2: בחירת קורס

בדף הבית תראה רשימה של קורסים.

כל קורס מציג:

- 📚 **שם הקורס** (לדוגמה: "מנועי בוכנה")
- 📝 **תיאור קצר** של הקורס
- 🟢 **כפתור "התחל"** להיכנס לקורס

צעד 3: צפייה בקורס ופרקים

בדף הקורס תראה:

עמודה ימנית (רשימת פרקים):

- 📖 רשימה של כל פרקי הקורס
- ✅ סטטוס של כל פרק (סיים / בתהליך)
- 📊 מספר הפרק (1, 2, 3...)

עמודה שמאלית (תוכן קורס):

- 🎓 כותרת הקורס
- 📄 תיאור הקורס
- 📈 כרטיס התקדמות:
  - אחוז ההשלמה (XX%)

- מספר פרקים שהושלמו (X מתוך Y)
- כפתור "המשך לקורס"

צעד 4: קריאת פרק

לחץ על פרק מהרשימה.

בדף הפרק תראה:

- 📖 **כותרת הפרק**
- 📝 **תיאור מלא** (עם עיצוב: בולד, אטליק, רשימות וכו')
- 🖼️ **תמונות** (אם הן קיימות)
- 📥 **קבצים להורדה** (PDF, Word, Excel וכו')
- ➡️ **כפתורי ניווט** (פרק הקודם / הבא)

צעד 5: פתרון בחינות

אם יש בחינות בקורס:

1. לחץ על **"בחינות"** בתפריט
2. בחר **בחינה** מהרשימה
3. **ענה על השאלות:**
   - ✅ רשות רבת-ברירה (בחר תשובה אחת)
   - ✅ בחירה מרובה (בחר כמה תשובות)
   - ✅ טקסט חופשי (כתוב תשובה)
4. לחץ **"שלח תשובות"**
5. קבל **ציון** ופרטי ההצלחה

מדריך המנהל

👨‍💼 כמה מנהל מנהל את המערכת

הפעלת פאנל ניהול

לאחר התחברות כמנהל (ת"ז: 999999999):

1. לחץ על **"ניהול"** בתפריט הראשי
2. בחר אחד מ:

- 📚 **קורסים** - יצירה, עריכה, מחיקה
- 📖 **פרקים** - הוספה, עריכה, מחיקה
- 📝 **תוכן** - הוספת טקסט, תמונות, קבצים
- ❓ **שאלות** - ניהול בחינות

✨ יצירת קורס חדש

שלבים:

1. בפאנל ניהול, בחר **"קורסים"**
2. לחץ כפתור **"הוסף קורס"**
3. מלא את הפרטים:

- **שם קורס** (חובה) — לדוגמה: "מנועי בוכנה"
- **תיאור** (חובה) — הסבר קצר על הקורס
- **מחסום בחירה** (אופציונלי) — תנאי כניסה

4. לחץ **"שמור"**

✅ **הקורס נוצר בהצלחה!**

📖 הוספת פרק לקורס

שלבים:

1. בחר **קורס** מהרשימה
2. לחץ **"הוסף פרק"**
3. מלא את הפרטים:
   - **שם הפרק** (חובה) — לדוגמה: "עיקרון הפעולה"
   - **תיאור** (חובה) — הסבר מפורט (משתמש בעורך טקסט עשיר!)
   - **סדר** (חובה) — מספר סידורי (1, 2, 3...)
4. לחץ **"שמור"**

✅ **הפרק נוצר!**

📝 הוספת תוכן חינוכי

שלבים:

1. בחר **פרק** מהרשימה
2. לחץ **"הוסף תוכן"**
3. בחר **סוג תוכן:**
   - 📄 **טקסט עשיר** - משתמש בעורך WYSIWYG
   - 🖼️ **תמונה** - Upload תמונה
   - 📑 **קובץ** - PDF, Word, Excel, PowerPoint

אם בחרת "טקסט עשיר":

1. כתוב את הטקסט בעורך
2. עצב עם כלי הסרגל (ראה סעיף "עורך הטקסט העשיר")
3. לחץ **"שמור"**

אם בחרת "קובץ":

1. לחץ **"בחר קובץ"**
2. בחר קובץ מהמחשב (עד 100 MB)
3. לחץ **"שמור"**

✅ **התוכן הוסף בהצלחה!**

עורך הטקסט העשיר

מה זה WYSIWYG?

"What You See Is What You Get = **WYSIWYG**" — עורך טקסט שמציג עיצוב בזמן אמת.

בלומדת ענף 71, העורך משתמש ב-**contentEditable API** של HTML5, בלי ספריות חיצוניות.

כלי הסרגל

סמל	שם	תיאור	דוגמה
B	Bold	בולד	**טקסט זה בולד**
I	Italic	אטליק	*טקסט זה אטליק*
U	Underline	קו תחתון	<u>קו תחתון</u>
H	Heading	כותרת H3	### כותרת גדולה
🎨	Color	בחר צבע	אדום
≡	Unordered	רשימה לא מסודרת	• פריט 1 • פריט 2
.1	Ordered	רשימה מסודרת	1. פריט 1 2. פריט 2
←	Align Left	יישור לשמאל	← יישור
→	Align Right	יישור לימין	יישור →
‖	Align Center	יישור למרכז	→ יישור ←
X	Clear	ניקוי עיצוב	טקסט רגיל

דוגמה: כתיבת פרק מלא

```
מנוע בוכנה – עיקרון הפעולה

(כדי לעשות זה כותרת H לחץ)

מנוע בוכנה הוא מנוע בעירה פנימית המשתמש בבוכנות וגלילים לחולל כוח.

יתרונות:
(בחר ≡ לחץ Unordered List)
• עלות נמוכה
• עמידות גבוהה לזמן
• דלק זול וקל להשגה
• טכנולוגיה מוכנת

חסרונות:
• זיהום אוויר גבוה
• קול רועם גבוה
• צריכת דלק גבוהה

(בחר טקסט ולחץ 🎨 לצבע)
```

HTML שנוצר בתוך המערכת

```
<h3>מנוע בוכנה – עיקרון הפעולה</h3>
<p>מנוע בוכנה הוא מנוע בעירה פנימית...</p>
<p><strong>יתרונות:</strong></p>
<ul>
  <li>עלות נמוכה</li>
  <li>עמידות גבוהה</li>
</ul>
```

Sanitization (אבטחה)

HTML שנכתב בעורך עובר דרך:

1. **sanitizeHtml()** - הסרת קודים מסוכנים
2. **whitelist validation** - רק תגים בטוחים
3. **content escaping** - הגנה מפני XSS

ניהול קבצים

סוגי קבצים הנתמכים

סוג	סיומות	גודל מקסימום	דוגמה
PDF	`pdf.`	100 MB	document.pdf
Word	`doc`, `.docx`	100 MB	guide.docx
Excel	`xls`, `.xlsx`	100 MB	data.xlsx
PowerPoint	`ppt`, `.pptx`	100 MB	slides.pptx
טקסט	`txt.`	100 MB	notes.txt
CSV	`csv.`	100 MB	spreadsheet.csv

Upload Flow

```
1. Admin קובץ" → "בוחר "הוסף תוכן"
   ↓
2. בוחר קובץ מהמחשב
   ↓
3. Frontend שולח ל-Backend דרך Multer
   ↓
4. Backend מתקדד:
   - בודק סוג קובץ (חומר בטוח)
   - בודק גודל (עד 100 MB)
   - שומר בתיקייה: /backend/uploads/
   - שומר נתיב בDB
   ↓
5. Frontend מציג אייקון וקובץ להורדה
   ↓
6. Students יכולים להוריד
```

Backend Upload Configuration

```
// src/config/upload.ts
export const allowedExtensions = /\.(jpe?
g|png|gif|webp|mp4|webm|ogg|pdf|docx?|xlsx?|pptx?|csv|txt)$/i;
export const maxFileSize = 100 * 1024 * 1024; // 100 MB

// Multer middleware
const upload = multer({
  dest: 'uploads/',
  fileFilter: (req, file, cb) => {
    if (allowedExtensions.test(file.originalname)) {
      cb(null, true);
    } else {
      cb(new Error('סוג קובץ לא תמוך'));
    }
  },
  limits: { fileSize: maxFileSize }
});
```

מדריך מפתח

🔧 עבודה מקומית

התקנה ראשונה

```
# 1. Clone / Download הפרויקט
cd "C:\Users\yairk\OneDrive\שולחן העבודה\rf-learning-system"

# 2. Install Backend dependencies
cd backend
npm install

# 3. Install Frontend dependencies
cd ../frontend
npm install

# 4. Setup Database
# ש וודא PostgreSQL רץ
# צור DB בשם: rf_learning
# הרץ backend/src/db.sql

# 5. Environment variables
# Backend: צור .env בתיקיית backend
# DB_HOST=localhost
# DB_PORT=5432
# DB_USER=postgres
# DB_PASSWORD=postgres123
# DB_NAME=rf_learning
# SERVER_PORT=3000
```

הפעלת Development Servers

```
# Terminal 1 - Backend
cd backend
npm run dev
# → Server running on http://localhost:3000

# Terminal 2 - Frontend
cd frontend
npm run dev
# → Local:   http://localhost:5173
```

Build for Production

```
# Frontend
cd frontend
npm run build
# → אפסקו לתיקייה: dist/

# Backend (just copy src to production)
cd backend
npm install --production
```

📁 קובץ מערכת (Directory Structure)

```
rf-learning-system/
├── backend/
│   ├── src/
│   │   ├── controllers/          # Business logic
│   │   │   ├── userController.ts
│   │   │   ├── courseController.ts
│   │   │   ├── chapterController.ts
│   │   │   ├── contentController.ts
│   │   │   ├── examController.ts
│   │   │   └── questionController.ts
│   │   ├── routes/               # API routes
│   │   │   ├── users.ts
│   │   │   ├── courses.ts
│   │   │   ├── chapters.ts
│   │   │   ├── content.ts
│   │   │   ├── exams.ts
│   │   │   └── questions.ts
│   │   ├── middleware/           # Middlewares
│   │   │   ├── auth.ts           # JWT verification
│   │   │   ├── errorHandler.ts   # Error handling
│   │   │   └── validation.ts     # Input validation
│   │   ├── config/               # Configuration
│   │   │   ├── database.ts       # DB connection pool
│   │   │   ├── upload.ts         # File upload rules
│   │   │   └── env.ts            # Environment vars
│   │   ├── utils/                # Helper functions
│   │   │   ├── validators.ts
│   │   │   └── formatters.ts
│   │   ├── server.ts             # Entry point
│   │   └── db.sql                # Database schema
│   ├── uploads/                  # Uploaded files
│   ├── package.json
│   ├── tsconfig.json
│   └── .env                      # Environment vars (not in git)
```

```
│
├── frontend/
│   ├── src/
│   │   ├── components/           # Reusable components
│   │   │   ├── Navbar.tsx
│   │   │   ├── Sidebar.tsx
│   │   │   ├── RichTextEditor.tsx
│   │   │   ├── ContentViewer.tsx
│   │   │   ├── ProgressCard.tsx
│   │   │   ├── CourseCard.tsx
│   │   │   └── ...
│   │   ├── pages/                # Full page components
│   │   │   ├── LoginPage.tsx
│   │   │   ├── HomePage.tsx
│   │   │   ├── CoursePage.tsx
│   │   │   ├── ChapterPage.tsx
│   │   │   ├── AdminPage.tsx
│   │   │   ├── AdminCoursesPage.tsx
│   │   │   ├── AdminChaptersPage.tsx
│   │   │   ├── AdminContentPage.tsx
│   │   │   └── ...
│   │   ├── context/              # React Context
│   │   │   ├── AuthContext.tsx   # Login/user state
│   │   │   └── ThemeContext.tsx  # Dark/Light mode
│   │   ├── services/             # API communication
│   │   │   └── api.ts            # Fetch wrappers
│   │   ├── types/                # TypeScript types
│   │   │   └── index.ts
│   │   ├── App.tsx               # Root component
│   │   ├── App.css               # Global styles
│   │   └── main.tsx              # Entry point
│   ├── index.html
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── dist/                     # Built files (after npm run
```

```
build)
│
├── screenshots/                    # צילומי מסך
├── START.bat                       # Automation script
├── SYSTEM_BOOK.md                  # Documentation
├── SYSTEM_BOOK.html
├── SYSTEM_BOOK.pdf
└── package.json (root)
```

API Examples 💻

Login

```
POST http://localhost:3000/api/auth/login

Body (JSON):
{
  "id_number": "123456789",
  "password": "password123"
}

Response:
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "John Student",
    "role": "student"
  }
}
```

Get All Courses

```
GET http://localhost:3000/api/courses
Headers: Authorization: Bearer <token>

Response:
[
  {
    "id": 1,
    "title": "מנועי בוכנה",
    "description": "קורס על מנועים...",
    "created_by": 1,
    "created_at": "2026-06-27T10:00:00Z"
  },
  ...
]
```

Create New Course (Admin only)

```
POST http://localhost:3000/api/courses
Headers: Authorization: Bearer <admin_token>

Body:
{
  "title": "קורס חדש",
  "description": "תיאור הקורס"
}

Response:
{
  "id": 10,
  "title": "קורס חדש",
  "description": "תיאור הקורס",
  "created_at": "2026-06-28T12:30:00Z"
}
```

Add Chapter

```
POST http://localhost:3000/api/chapters
Headers: Authorization: Bearer <admin_token>

Body:
{
  "course_id": 1,
  "title": "פרק חדש",
  "description": "תיאור הפרק",
  "sort_order": 1
}
```

Upload File

```
POST http://localhost:3000/api/upload
Headers: Authorization: Bearer <admin_token>
Content-Type: multipart/form-data

Body: (form-data)
file: <binary file data>
chapter_id: 1

Response:
{
  "filename": "1782324662605-281066631.pdf",
  "path": "/uploads/1782324662605-281066631.pdf",
  "originalName": "document.pdf"
}
```

CSS Architecture 🎨

```
/* Global variables - src/App.css */

:root {
  /* Light mode colors */
  --bg: #ffffff;
  --text: #1c1d1f;
  --primary: #a435f0;
  --border: #d1d7dc;
  --success: #1e8e3e;
  --error: #d33b27;
}

[data-theme="dark"] {
  --bg: #1c1d1f;
  --text: #f0f0f0;
  --primary: #c088f5;
  --border: #4a4d4f;
}

/* Usage in components */
body {
  background: var(--bg);
  color: var(--text);
}

button {
  background: var(--primary);
  color: white;
}
```

Authentication Flow 🔐

```
1. User enters ID + Password
   ↓
2. Frontend POST /api/auth/login
   ↓
3. Backend:
   - בודק user בDB
   - משווה password עם bcrypt
   - מייצר JWT token
   ↓
4. Frontend stores token בlocalStorage
   ↓
5. כל בקשה עתידית תכלול:
   Headers: Authorization: Bearer <token>
   ↓
6. Backend מאמת token עם JWT middleware
   ↓
7. אם תקין → גישה להמשך
   אם לא → error 401 Unauthorized
```

שאלות נפוצות

❓ "איזה גרסת Node.js צריך?"

תשובה: Node.js 16 ומעלה (בדוק עם `node --version`)

❓ ""Database rf_learning not found" אני מקבל"

פתרון:

1. פתח pgAdmin
2. לחץ ימני על "Databases"
3. בחר "Create" → "Database"
4. שם: `rf_learning`, Owner: `postgres`
5. הרץ את `backend/src/db.sql`

❓ "Port 3000/5173 already in use"

פתרון:

```
taskkill /F /IM node.exe
# ואז הרץ שוב את START.bat
```

❓ "איך אני מוסיף משתמש חדש?"

פתרון: עכשיו צריך פתוח מערכת (אין UI). בעתיד נוסיף admin panel לניהול משתמשים.

❓ "היכן הקבצים שהעלאתי מאוחסנים?"

פתרון: בתיקייה `/backend/uploads` בתוך השרת. לא זמין לרשת (נדרש להוצא לcombine public uploads folder).

❓ "איך אני מוציא את המערכת לProduction?"

תשובה: צריך:

1. Server עם Node.js וPostgreSQL
2. Nginx / Apache כ-reverse proxy
3. SSL certificate (https)
4. PM2 לריצת backend בbackground
5. Environment variables (.env)

🎓 סיכום

לומדת ענף 71 היא מערכת למידה **מלאה וחזקה**:

✨ תכונות עיקריות:

- ✅ Architecture מודרנית Tier-3
- ✅ Frontend עם React + TypeScript + Vite
- ✅ Backend חזק עם Express + Node.js
- ✅ Database relational עם PostgreSQL
- ✅ API RESTful מלאה
- ✅ File upload עם support לקבצים גדולים
- ✅ Rich text editor בלי ספריות חיצוניות
- ✅ Light/Dark mode
- ✅ RTL Hebrew support

🚀 מוכנה לשימוש:

- 👩‍🎓 תלמידים יכולים ללמוד בקלות
- 👨‍💼 מנהלים יכולים ליצור תוכן בקלות
- 🔒 אבטחה ו-validation כחוק
- 📱 Responsive על כל המכשירים

עדכון אחרון: 2026-06-28

גרסה: 1.0

שפה: TypeScript + JavaScript

סטטוס: Production Ready

ספר המערכת - לומדת ענף 71

עדכון אחרון: 2026-06-28

גרסה: Extended 1.0